→ Decimal to Binary :-

↓

0-9

(1-0)

87 ⟷ 0|0|1|1|1

2	20	
2	10	0
2	5	0
2	2	1
	1	0

(1|0|0|0)

13

2	13	
2	6	1
2	3	0
2	1	1

1101

1000 +

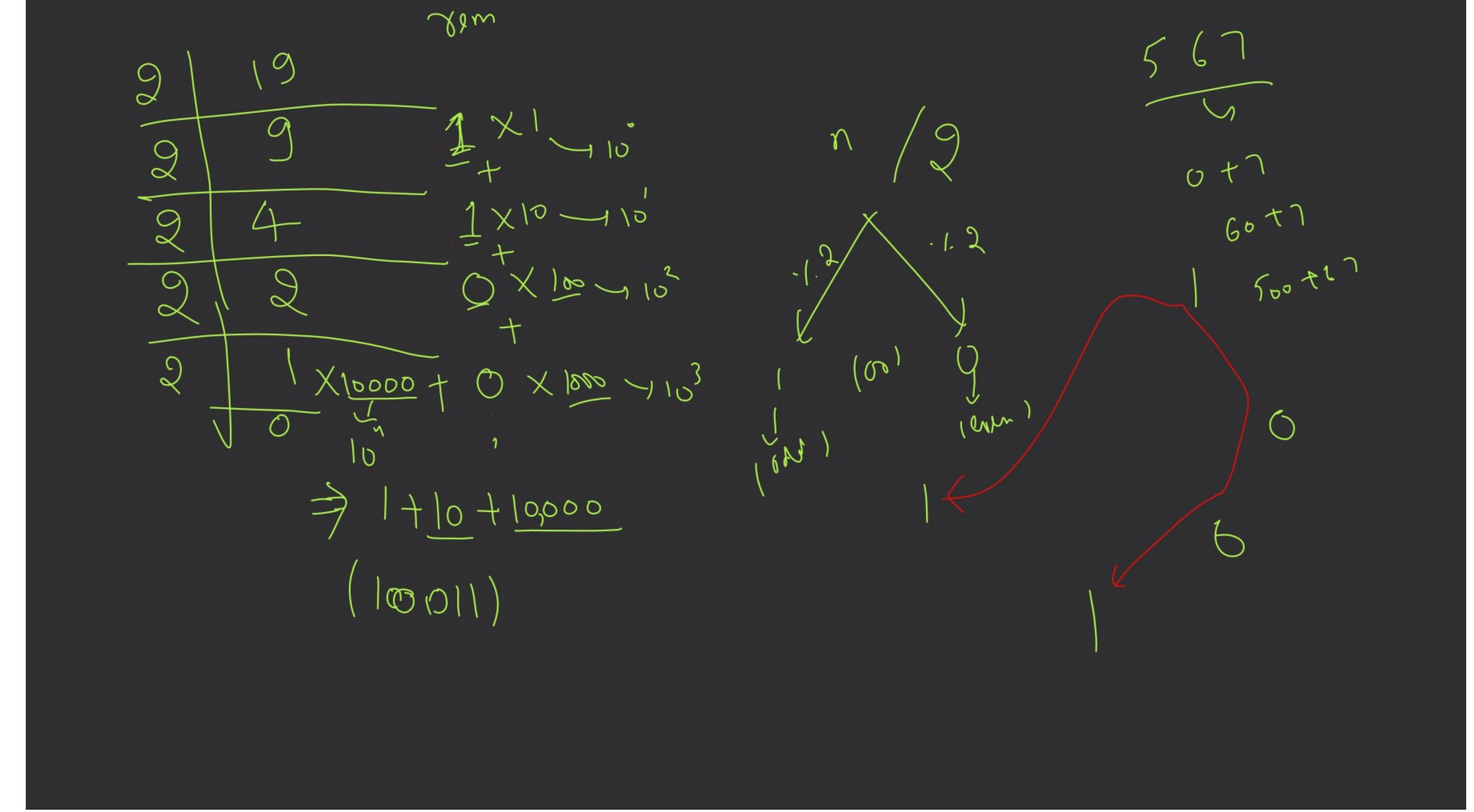
1×1 + 0×10 + 1×100 + 1×1000

1 × 1 = 1 + 0 +

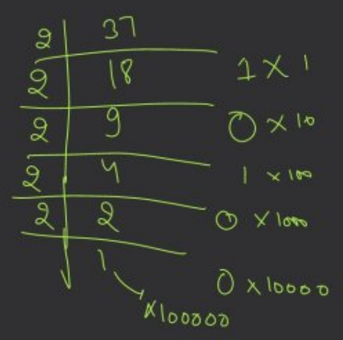
1 × 10 + 100 + 1000

1 × 100 = 1101

1
10
100
1000
+

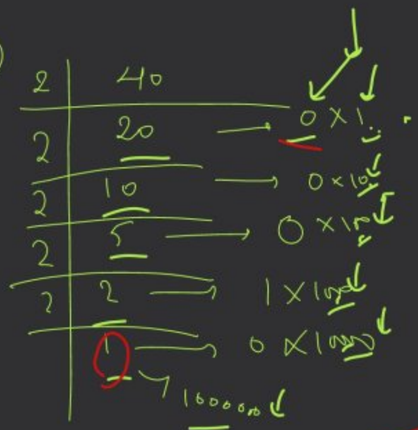

2 | 19
2 | 9 — 1 × 1 → 10^0
2 | 4 — 1 × 10 → 10^1
2 | 2 — 0 × 100 → 10^2
2 | 1 — 1 × 10000 → 10^4
2 | 0 — 0 × 1000 → 10^3
⇒ 1 + 10 + 10,000
(10011)
n / 2
÷ 2
÷ 2
1
0
1
5 6 7
0 + 7
60 + 7
500 + 7
1
0
0
1

(37)



100101 Ans

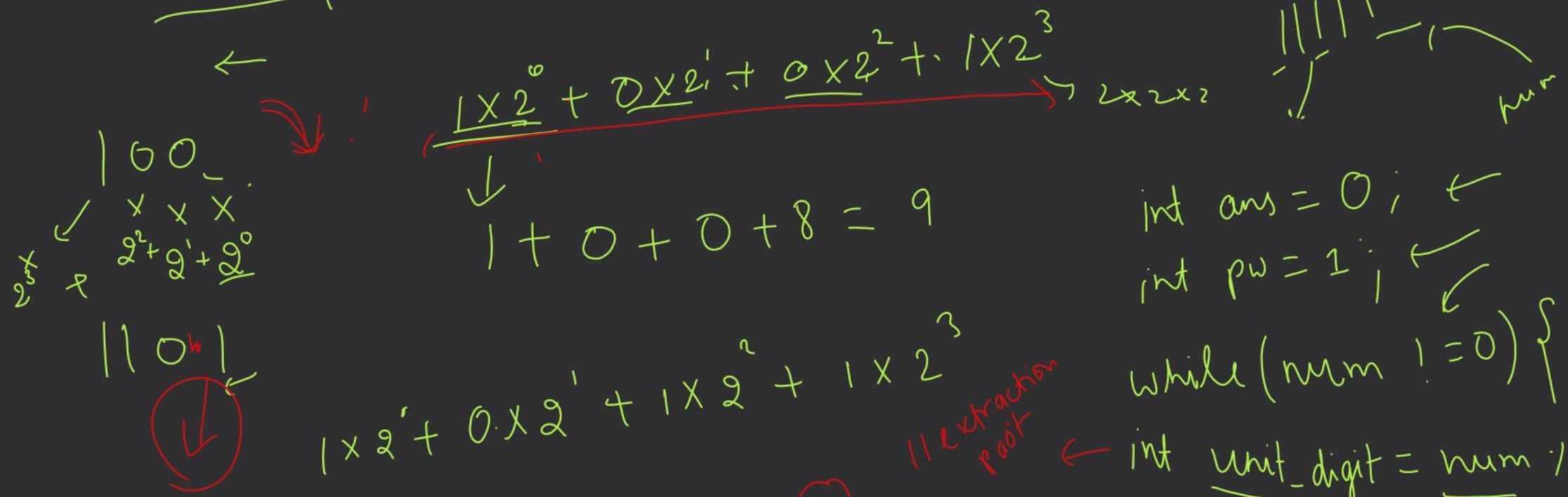
(40)



47

```
int ans = 0;
int pw = 1;
While (n != 0) {
int rem = n % 2;
ans = ans + (rem * pw);
n = n / 2;
pw = pw * 10;
}
```

Binary to Decimal :-




```
int ans = 0;
int pw = 1;
while(num != 0){
int unit_digit = num % 10;   //extraction part
ans = ans + (unit_digit * pw);
num = num / 10;              //elimination part
pw = pw * 2;
}
```

Type Casting :-

- Implicit
- Explicit

```
int a = 10;
double b = a;
```

int → double

b

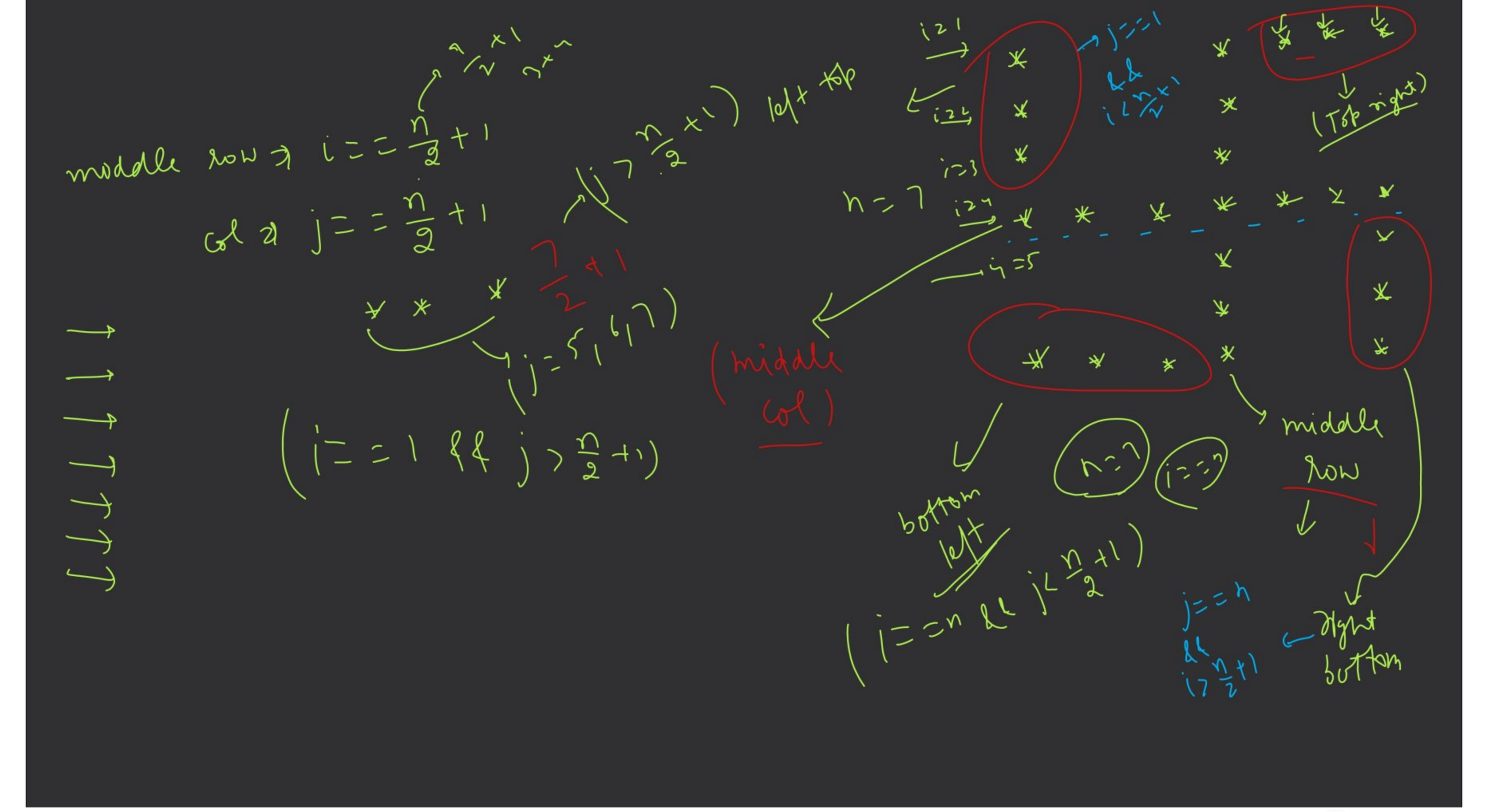
10.0

```
double a = 10.7;
int b = (int) a;
```

10.7

10

(integral part of a)



```
n=7;
int i=1;
while(i<=n){
int j=1;
while(j<=n){
String ch=" ";
if(i==n/2+1)
ch="*";
else if(j==n/2+1)
ch="*";
else if(i==1 && j>n/2+1)
ch="*";
else if(j==1 && i<n/2+1)
ch="*";
else if(i==n && j<n/2+1)
ch="*";
else if(j==n && i>n/2+1) ch="*";
Sout(ch);
j++;
Soutpln();
i++;
```

Object Oriented Programming (OOP) in Java – Beginner Notes

What is OOP?

OOP (Object Oriented Programming) is a programming paradigm where we structure programs using objec
It helps in organizing code into reusable components and models real world entities.

Languages using OOP include Java, C++, Python, and C#.

1. Class

A class is a blueprint used to create objects. It defines variables (data) and methods (functions).

Example:

```
class Student {
    String name;
    int age;

    void study(){
        System.out.println("Student is studying");
    }
}
```

2. Object

An object is an instance of a class. It is used to access the variables and methods of the class.

```
    }
}
```

2. Object

An object is an instance of a class. It is used to access the variables and methods of the class.

```
Example:
Student s1 = new Student();
s1.name = "Rahul";
s1.age = 20;
s1.study();
```

3. Encapsulation

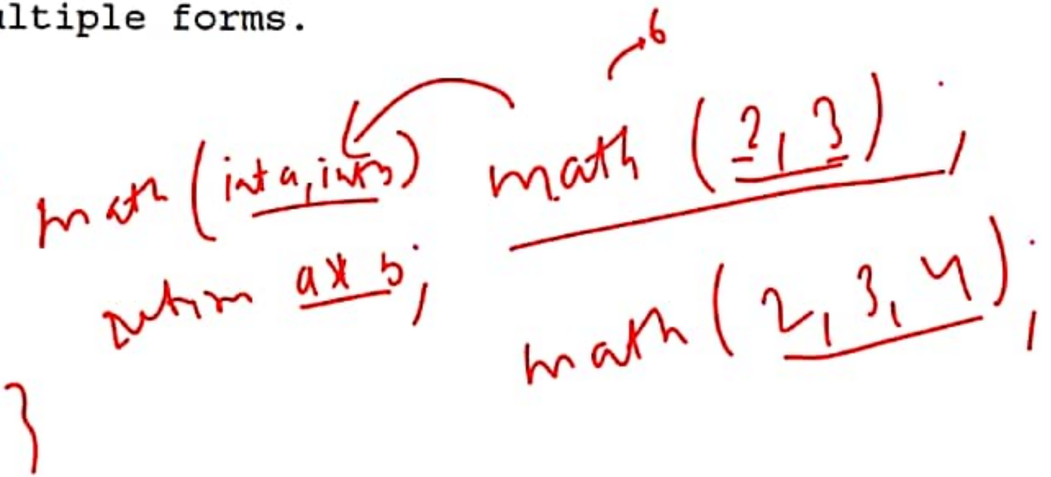
Encapsulation means wrapping data and methods together and protecting the data from outside access.
It is achieved using private variables and public getter/setter methods.

```
Example:
class Person {
    private int age;

    public void setAge(int age){
        this.age = age;
    }

    public int getAge(){
        return age;
    }
}
```

5. Polymorphism

Polymorphism means one method having multiple forms.

```
Method Overloading (Compile Time):
class MathUtil {
    int add(int a, int b){
        return a+b;
    }
    int add(int a, int b, int c){
        return a+b+c;
    }
}
```




```
Method Overriding (Runtime):
class Animal {
    void sound(){
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal {
    void sound(){
        System.out.println("Dog barks");
    }
}
```